

ESCUELA POLITÉCNICA NACIONAL

Facultad de Sistemas

Ingeniería en Ciencias de la Computación

Nombre del profesor: Mg. Marco Aguirre

Nombre: Justin Cusquillo

Curso: GR1CC

Fecha de entrega: 13 de noviembre del 2026

Laboratorio 4

"E SCIENTIA HOMINIS SALUS"

1. Introducción

Cuando varios hilos comparten recursos, como un archivo o un contador, pueden aparecer errores porque el orden de ejecución cambia en cada corrida, para evitarlo se usan mecanismos de sincronización como los semáforos y los mutex y en este laboratorio se aplicaron en Python con la biblioteca threading en tres casos: tres agentes de IA que comparten un LLM en un orden fijo, la escritura con y sin exclusión mutua, y un sistema de lectores y escritores sobre un archivo

2. Objetivos

- Sincronizar tres hilos para que accedan a un recurso en el orden A1, A3, A2
- Comparar la escritura concurrente sin y con exclusión mutua
- Implementar lectores y escritores sobre BASE.txt con lecturas simultáneas y escrituras exclusivas

3. Mitología y procedimiento

Problema 1: hilos y sincronización de agentes de IA mediante semáforos

Actividad 1: acceso ordenado a un LLM compartido

Implementación con hilos en Python

1. Implemente el diseño en 00_agentes_ia.py, utilizando threading.Thread y threading.Semaphore. Cree los semáforos una sola vez y compártalos entre los tres hilos.

```
 GNU nano 8.7.1                                                    00_agentes_ia.py *
"""Actividad 1: tres agentes de IA (A1 -> A3 -> A2) comparten un LLM
"""
import threading, time, random, sys

CICLOS = 5 # variante finita (None = repeticion indefinida)
T_MAX = float(sys.argv[2]) if len(sys.argv) > 2 else 0.3

# Semaforos creados UNA sola vez y compartidos. Valores iniciales:
# s1=1 -> A1 arranca primero; s3=0 y s2=0 -> A3 y A2 esperan.
s1 = threading.Semaphore(1)
s3 = threading.Semaphore(0)
s2 = threading.Semaphore(0)

def usar_llm(nombre, ciclo):
    print(f"[{nombre}] Ciclo {ciclo}: INICIA acceso al LLM", flush=True)
    time.sleep(random.uniform(0.05, T_MAX))   # simula tiempo de respuesta
    print(f"[{nombre}] Ciclo {ciclo}: TERMINA acceso al LLM", flush=True)

def agente(nombre, mi_sem, sig_sem):
    ciclo = 1
    while CICLOS is None or ciclo <= CICLOS:
        mi_sem.acquire()          # wait(): espera mi turno
        usar_llm(nombre, ciclo)
        sig_sem.release()         # signal(): habilita al siguiente SOLO al terminar
        ciclo += 1

if __name__ == "__main__":
    hilos = {
        "A1": threading.Thread(target=agente, args=("A1", s1, s3)),
        "A3": threading.Thread(target=agente, args=("A3", s3, s2)),
        "A2": threading.Thread(target=agente, args=("A2", s2, s1)),
    }
    orden = (sys.argv[1] if len(sys.argv) > 1 else "A1,A3,A2").split(",")
    print(f"Orden de inicio de hilos: {orden}\n")
    for n in orden:
        hilos[n].start()
    for h in hilos.values():
        h.join()
    print("\nFin: 15 intervenciones completas (5 ciclos).")
```

2. Defina una función para simular el acceso al LLM. Muestre el inicio y fin de cada intervención, el nombre del agente y el número de ciclo. Use una pausa breve para representar el tiempo de respuesta. No se requiere un LLM real, conexión a Internet ni claves de API.

```
def usar_llm(nombre, ciclo):
    print(f"[{nombre}] Ciclo {ciclo}: INICIA acceso al LLM", flush=True)
    time.sleep(random.uniform(0.05, T_MAX))   # simula tiempo de respuesta
    print(f"[{nombre}] Ciclo {ciclo}: TERMINA acceso al LLM", flush=True)
```

3. Inicie los tres hilos antes de esperar su terminación. El orden de las llamadas a start() no debe determinar el orden de acceso al LLM: esa responsabilidad corresponde a los semáforos.

```
if __name__ == "__main__":
    hilos = {
        "A1": threading.Thread(target=agente, args=("A1", s1, s3)),
        "A3": threading.Thread(target=agente, args=("A3", s3, s2)),
        "A2": threading.Thread(target=agente, args=("A2", s2, s1)),
    }
    orden = (sys.argv[1] if len(sys.argv) > 1 else "A1,A3,A2").split(",")
    print(f"Orden de inicio de hilos: {orden}\n")
    for n in orden:
        hilos[n].start()
    for h in hilos.values():
        h.join()
    print("\nFin: 15 intervenciones completas (5 ciclos).")
```

4. Mantenga el pseudocódigo con repetición indefinida. Para obtener una ejecución finita y verificable en Python, pruebe una variante de cinco ciclos completos, con el mismo número de intervenciones por agente, y espere a los tres hilos con join().

```
justin@ubuntu-server:~/lab4$ python3 00_agentes_ia.py A1,A3,A2
Orden de inicio de hilos: ['A1', 'A3', 'A2']

[A1] Ciclo 1: INICIA acceso al LLM
[A1] Ciclo 1: TERMINA acceso al LLM
[A3] Ciclo 1: INICIA acceso al LLM
[A3] Ciclo 1: TERMINA acceso al LLM
[A2] Ciclo 1: INICIA acceso al LLM
[A2] Ciclo 1: TERMINA acceso al LLM
[A1] Ciclo 2: INICIA acceso al LLM
[A1] Ciclo 2: TERMINA acceso al LLM
[A3] Ciclo 2: INICIA acceso al LLM
[A3] Ciclo 2: TERMINA acceso al LLM
[A2] Ciclo 2: INICIA acceso al LLM
[A2] Ciclo 2: TERMINA acceso al LLM
[A1] Ciclo 3: INICIA acceso al LLM
[A1] Ciclo 3: TERMINA acceso al LLM
[A3] Ciclo 3: INICIA acceso al LLM
[A3] Ciclo 3: TERMINA acceso al LLM
[A2] Ciclo 3: INICIA acceso al LLM
[A2] Ciclo 3: TERMINA acceso al LLM
[A1] Ciclo 4: INICIA acceso al LLM
[A1] Ciclo 4: TERMINA acceso al LLM
[A3] Ciclo 4: INICIA acceso al LLM
[A3] Ciclo 4: TERMINA acceso al LLM
[A2] Ciclo 4: INICIA acceso al LLM
[A2] Ciclo 4: TERMINA acceso al LLM
[A1] Ciclo 5: INICIA acceso al LLM
[A1] Ciclo 5: TERMINA acceso al LLM
[A3] Ciclo 5: INICIA acceso al LLM
[A3] Ciclo 5: TERMINA acceso al LLM
[A2] Ciclo 5: INICIA acceso al LLM
[A2] Ciclo 5: TERMINA acceso al LLM

Fin: 15 intervenciones completas (5 ciclos).
justin@ubuntu-server:~/lab4$
```

5. Repita la prueba cambiando el orden de inicio de los hilos y los tiempos de acceso simulados. Verifique que la secuencia de acceso siga siendo A1, A3, A2.

```
justin@ubuntu-server:~/lab4$ python3 00_agentes_ia.py A2,A3,A1 0.8
Orden de inicio de hilos: ['A2', 'A3', 'A1']

[A1] Ciclo 1: INICIA acceso al LLM
[A1] Ciclo 1: TERMINA acceso al LLM
[A3] Ciclo 1: INICIA acceso al LLM
[A3] Ciclo 1: TERMINA acceso al LLM
[A2] Ciclo 1: INICIA acceso al LLM
[A2] Ciclo 1: TERMINA acceso al LLM
[A1] Ciclo 2: INICIA acceso al LLM
[A1] Ciclo 2: TERMINA acceso al LLM
[A3] Ciclo 2: INICIA acceso al LLM
[A3] Ciclo 2: TERMINA acceso al LLM
[A2] Ciclo 2: INICIA acceso al LLM
[A2] Ciclo 2: TERMINA acceso al LLM
[A1] Ciclo 3: INICIA acceso al LLM
[A1] Ciclo 3: TERMINA acceso al LLM
[A3] Ciclo 3: INICIA acceso al LLM
[A3] Ciclo 3: TERMINA acceso al LLM
[A2] Ciclo 3: INICIA acceso al LLM
[A2] Ciclo 3: TERMINA acceso al LLM
[A1] Ciclo 4: INICIA acceso al LLM
[A1] Ciclo 4: TERMINA acceso al LLM
[A3] Ciclo 4: INICIA acceso al LLM
[A3] Ciclo 4: TERMINA acceso al LLM
[A2] Ciclo 4: INICIA acceso al LLM
[A2] Ciclo 4: TERMINA acceso al LLM
[A1] Ciclo 5: INICIA acceso al LLM
[A1] Ciclo 5: TERMINA acceso al LLM
[A3] Ciclo 5: INICIA acceso al LLM
[A3] Ciclo 5: TERMINA acceso al LLM
[A2] Ciclo 5: INICIA acceso al LLM
[A2] Ciclo 5: TERMINA acceso al LLM

Fin: 15 intervenciones completas (5 ciclos).
justin@ubuntu-server:~/lab4$
```

Actividad de consulta

Explique como utilizar un LLM local en su computadora requisitos(maximo una hoja)

Un Modelo de Lenguaje de Gran Tamaño (LLM) local permite ejecutar inferencias directamente en el hardware del equipo sin enviar datos a servidores externos, garantizando privacidad, nula latencia de red y funcionamiento sin conexión a Internet

Requisitos:

- Sistema operativo: Windows, macOS o Linux (Ubuntu)
- RAM: mínimo 8 GB para modelos pequeños ; 16 GB recomendados para modelo de 7B a 8B
- Procesador: CPU moderna de 64 bits. Funciona sin GPU, pero más lento.
- GPU (opcional): una tarjeta NVIDIA con 6 a 8 GB de VRAM o más acelera mucho la respuesta.
- Almacenamiento: entre 2 y 10 GB libres por modelo, según su tamaño.
- Internet: solo para descargar la herramienta y el modelo; después puede usarse sin conexión.

Pasos con Ollama:

1. Instalar Ollama (en Linux: curl -fsSL https://ollama.com/install.sh | sh).
2. Descargar y ejecutar un modelo: ollama run llama3.2.

3. Escribir las preguntas directamente en la terminal.
4. Para salir, escribir /bye.

Ventajas: privacidad, uso sin conexión y sin costo por uso

Limitaciones: exige hardware, y los modelos pequeños son menos precisos que los modelos en la nube

Actividad : SIN MUTEX

```
  GNU nano 8.7.1                                                    01_sin_mutex.py
"""Escritura SIN exclusion mutua: condicion de carrera sobre un contador
y sobre un archivo compartido."""
import threading, time

N_HILOS, N_ITER = 4, 100
contador = 0

def incrementar():
    global contador
    for _ in range(N_ITER):
        tmp = contador          # leer
        time.sleep(0.0001)      # fuerza cambio de contexto
        contador = tmp + 1      # escribir (puede pisar a otro hilo)

def escribir_archivo(id_hilo):
    with open("salida_sin_mutex.txt", "a") as f:
        for i in range(5):
            f.write(f"Hilo {id_hilo} escribe parte ")
            f.flush(); time.sleep(0.01)
            f.write(f"{i}\n")

if __name__ == "__main__":
    open("salida_sin_mutex.txt", "w").close()
    hs = [threading.Thread(target=incrementar) for _ in range(N_HILOS)]
    [h.start() for h in hs]; [h.join() for h in hs]
    print(f"Contador esperado: {N_HILOS*N_ITER} | obtenido: {contador}")
    hs = [threading.Thread(target=escribir_archivo, args=(i,)) for i in range(N_HILOS)]
    [h.start() for h in hs]; [h.join() for h in hs]
    print("Revise salida_sin_mutex.txt: las lineas aparecen mezcladas.")
```

```
justin@ubuntu-server:~/lab4$ python3 01_sin_mutex.py
Contador esperado: 400 | obtenido: 100
Revise salida_sin_mutex.txt: las lineas aparecen mezcladas.
justin@ubuntu-server:~/lab4$ cat salida_sin_mutex.txt
Hilo 0 escribe parte Hilo 1 escribe parte Hilo 2 escribe parte Hilo 3 escribe parte 0
Hilo 0 escribe parte 0
Hilo 1 escribe parte 0
Hilo 2 escribe parte 0
Hilo 3 escribe parte 1
Hilo 3 escribe parte 1
Hilo 1 escribe parte 1
Hilo 2 escribe parte 1
Hilo 0 escribe parte 2
Hilo 2 escribe parte 2
Hilo 1 escribe parte 2
Hilo 0 escribe parte 2
Hilo 3 escribe parte 3
Hilo 0 escribe parte 3
Hilo 1 escribe parte 3
Hilo 3 escribe parte 3
Hilo 2 escribe parte 4
4
4
4
justin@ubuntu-server:~/lab4$
```

CON MUTEX

```
  GNU nano 8.7.1                                                    02_con_mutex.py
"""Escritura CON exclusion mutua (Lock/Semaphore(1)): resultado correcto."""
import threading, time

N_HILOS, N_ITER = 4, 100
contador = 0
mutex = threading.Semaphore(1)   # equivalente a un mutex

def incrementar():
    global contador
    for _ in range(N_ITER):
        mutex.acquire()
        tmp = contador
        time.sleep(0.0001)
        contador = tmp + 1
        mutex.release()

def escribir_archivo(id_hilo):
    for i in range(5):
        with mutex:
            with open("salida_con_mutex.txt", "a") as f:
                f.write(f"Hilo {id_hilo} escribe parte ")
                f.flush(); time.sleep(0.01)
                f.write(f"{i}\n")

if __name__ == "__main__":
    open("salida_con_mutex.txt", "w").close()
    hs = [threading.Thread(target=incrementar) for _ in range(N_HILOS)]
    [h.start() for h in hs]; [h.join() for h in hs]
    print(f"Contador esperado: {N_HILOS*N_ITER} | obtenido: {contador}")
    hs = [threading.Thread(target=escribir_archivo, args=(i,)) for i in range(N_HILOS)]
    [h.start() for h in hs]; [h.join() for h in hs]
    print("Revise salida_con_mutex.txt: cada linea esta completa y ordenada.")
```

```
justin@ubuntu-server:~/lab4$ python3 02_con_mutex.py
Contador esperado: 400 | obtenido: 400
Revise salida_con_mutex.txt: cada linea esta completa y ordenada.
justin@ubuntu-server:~/lab4$ cat salida_con_mutex.txt
Hilo 0 escribe parte 0
Hilo 0 escribe parte 1
Hilo 0 escribe parte 2
Hilo 0 escribe parte 3
Hilo 0 escribe parte 4
Hilo 2 escribe parte 0
Hilo 2 escribe parte 1
Hilo 2 escribe parte 2
Hilo 2 escribe parte 3
Hilo 2 escribe parte 4
Hilo 3 escribe parte 0
Hilo 3 escribe parte 1
Hilo 3 escribe parte 2
Hilo 3 escribe parte 3
Hilo 3 escribe parte 4
Hilo 1 escribe parte 0
Hilo 1 escribe parte 1
Hilo 1 escribe parte 2
Hilo 1 escribe parte 3
Hilo 1 escribe parte 4
```

Problema 2: lectores y escritores mediante semáforos

Actividad 5: implementación y ejecución en Python

Guarde su solución como 03_lectores_escritores.py. Utilice las clases threading.Thread y threading.Semaphore.

GNU nano 8.7.1 03_lectores_escritores.py *

```python
"""Problema 2: lectores y escritores con semaforos sobre BASE.txt.
"""
import threading, time, random, sys

ARCHIVO = "BASE.txt"
mutex_lectores = threading.Semaphore(1)  # protege 'lectores_activos'
acceso_archivo = threading.Semaphore(1)  # exclusion lector/escritor
lectores_activos = 0
t0 = time.time()

def log(msg):
    print(f"[{time.time()-t0:5.2f}s] {msg}", flush=True)

def lector(id_):
    global lectores_activos
    time.sleep(random.uniform(0, 1.5))
    mutex_lectores.acquire()
    lectores_activos += 1
    if lectores_activos == 1:          # el primero bloquea a los escritores
        acceso_archivo.acquire()
    mutex_lectores.release()
    try:
        log(f"Lector {id_}: INICIA lectura (activos={lectores_activos})")
        with open(ARCHIVO, "r") as f:
            n = len(f.readlines())
        time.sleep(random.uniform(0.5, 1.0))
        log(f"Lector {id_}: TERMINA lectura ({n} lineas)")
    finally:
        mutex_lectores.acquire()
        lectores_activos -= 1
        if lectores_activos == 0:      # el ultimo libera a los escritores
            acceso_archivo.release()
        mutex_lectores.release()

def escritor(id_):
    time.sleep(random.uniform(0, 1.5))
    acceso_archivo.acquire()
    try:
        log(f"Escritor {id_}: INICIA escritura (ACCESO EXCLUSIVO)")
        with open(ARCHIVO, "r") as f:
            f.read()                   # consulta
        with open(ARCHIVO, "a") as f:
            f.write(f"Registro del Escritor {id_} - operacion: ESCRITURA\n")
        time.sleep(random.uniform(0.5, 1.0))
        log(f"Escritor {id_}: TERMINA escritura")
    finally:
        acceso_archivo.release()
```

```python
def escritor(id_):
    time.sleep(random.uniform(0, 1.5))
    acceso_archivo.acquire()
    try:
        log(f"Escritor {id_}: INICIA escritura (ACCESO EXCLUSIVO)")
        with open(ARCHIVO, "r") as f:
            f.read()                   # consulta
        with open(ARCHIVO, "a") as f:
            f.write(f"Registro del Escritor {id_} - operacion: ESCRITURA\n")
        time.sleep(random.uniform(0.5, 1.0))
        log(f"Escritor {id_}: TERMINA escritura")
    finally:
        acceso_archivo.release()

if __name__ == "__main__":
    nl = int(sys.argv[1]) if len(sys.argv) > 1 else 3
    ne = int(sys.argv[2]) if len(sys.argv) > 2 else 2
    with open(ARCHIVO, "w") as f:      # inicializar UNA vez
        f.write("=== BASE DE DATOS ===\nRegistro 1: Ana\nRegistro 2: Luis\n")
    hs = [threading.Thread(target=lector, args=(i+1,)) for i in range(nl)]
    hs += [threading.Thread(target=escritor, args=(i+1,)) for i in range(ne)]
    random.shuffle(hs)
    [h.start() for h in hs]; [h.join() for h in hs]
    print("\n--- Contenido final de BASE.txt ---")
    print(open(ARCHIVO).read())
```

1. Inicialice BASE.txt una sola vez, antes de iniciar los hilos, con un encabezado o varios registros de prueba.

```
justin@ubuntu-server:~/lab4$ cat BASE.txt
=== BASE DE DATOS ===
Registro 1: Ana
Registro 2: Luis
Registro del Escritor 2 - operacion: ESCRITURA
Registro del Escritor 1 - operacion: ESCRITURA
justin@ubuntu-server:~/lab4$
```

```
if __name__ == "__main__":
    nl = int(sys.argv[1]) if len(sys.argv) > 1 else 3
    ne = int(sys.argv[2]) if len(sys.argv) > 2 else 2
    with open(ARCHIVO, "w") as f:      # inicializar UNA vez
        f.write("=== BASE DE DATOS ===\nRegistro 1: Ana\nRegistro 2: Luis\n")
    hs = [threading.Thread(target=lector, args=(i+1,)) for i in range(nl)]
    hs += [threading.Thread(target=escritor, args=(i+1,)) for i in range(ne)]
    random.shuffle(hs)
    [h.start() for h in hs]; [h.join() for h in hs]
    print("\n--- Contenido final de BASE.txt ---")
    print(open(ARCHIVO).read())
```

2. Cree por lo menos tres hilos lectores y dos hilos escritores. Las funciones deben estar claramente separadas.

```
def lector(id_):
    global lectores_activos
    time.sleep(random.uniform(0, 1.5))
    mutex_lectores.acquire()
    lectores_activos += 1
    if lectores_activos == 1:          # el primero bloquea a los escritores
        acceso_archivo.acquire()
    mutex_lectores.release()
    try:
        log(f"Lector {id_}: INICIA lectura (activos={lectores_activos})")
        with open(ARCHIVO, "r") as f:
            n = len(f.readlines())
        time.sleep(random.uniform(0.5, 1.0))
        log(f"Lector {id_}: TERMINA lectura ({n} lineas)")
    finally:
        mutex_lectores.acquire()
        lectores_activos -= 1
        if lectores_activos == 0:      # el ultimo libera a los escritores
            acceso_archivo.release()
        mutex_lectores.release()

def escritor(id_):
    time.sleep(random.uniform(0, 1.5))
    acceso_archivo.acquire()
    try:
        log(f"Escritor {id_}: INICIA escritura (ACCESO EXCLUSIVO)")
        with open(ARCHIVO, "r") as f:
            f.read()                   # consulta
        with open(ARCHIVO, "a") as f:
            f.write(f"Registro del Escritor {id_} - operacion: ESCRITURA\n")
        time.sleep(random.uniform(0.5, 1.0))
        log(f"Escritor {id_}: TERMINA escritura")
    finally:
        acceso_archivo.release()
```

3. Cada lector deberá abrir y leer el contenido real del archivo. Cada escritor deberá agregar un registro que identifique al hilo y su operación.

```
 with open(ARCHIVO, "r") as f:
     n = len(f.readlines())
 time.sleep(random.uniform(0.5, 1.0))
print(f"Lector {id_}: TERMINA lectura ({rel_lineas} lineas)")
with open(ARCHIVO, "r") as f:
    f.read()                    # consulta
with open(ARCHIVO, "a") as f:
    f.write(f"Registro del Escritor {id_} - operacion: ESCRITURA\n")
```

4. Muestre en consola el inicio y la finalización de cada lectura o escritura. Introduzca pausas controladas con time.sleep() durante el acceso para observar la concurrencia.

```
justin@ubuntu-server:~/lab4$ python3 03_lectores_escritores.py 3 2
[ 0.05s] Lector 2: INICIA lectura (activos=1)
[ 0.57s] Lector 3: INICIA lectura (activos=2)
[ 0.65s] Lector 1: INICIA lectura (activos=3)
[ 0.75s] Lector 2: TERMINA lectura (3 lineas)
[ 1.53s] Lector 3: TERMINA lectura (3 lineas)
[ 1.64s] Lector 1: TERMINA lectura (3 lineas)
[ 1.64s] Escritor 1: INICIA escritura (ACCESO EXCLUSIVO)
[ 2.44s] Escritor 1: TERMINA escritura
[ 2.44s] Escritor 2: INICIA escritura (ACCESO EXCLUSIVO)
[ 3.42s] Escritor 2: TERMINA escritura

--- Contenido final de BASE.txt ---
=== BASE DE DATOS ===
Registro 1: Ana
Registro 2: Luis
Registro del Escritor 1 - operacion: ESCRITURA
Registro del Escritor 2 - operacion: ESCRITURA
```

5. Emplee semáforos para garantizar que las lecturas puedan superponerse entre sí, pero nunca con una escritura. Verifique los resultados mediante la salida de consola y el contenido final del archivo.

```
[ 0.05s] Lector 2: INICIA lectura (activos=1)
[ 0.57s] Lector 3: INICIA lectura (activos=2)
[ 0.65s] Lector 1: INICIA lectura (activos=3)
[ 0.75s] Lector 2: TERMINA lectura (3 lineas)
[ 1.53s] Lector 3: TERMINA lectura (3 lineas)
[ 1.64s] Lector 1: TERMINA lectura (3 lineas)
[ 1.64s] Escritor 1: INICIA escritura (ACCESO EXCLUSIVO)
[ 2.44s] Escritor 1: TERMINA escritura
[ 2.44s] Escritor 2: INICIA escritura (ACCESO EXCLUSIVO)
[ 3.42s] Escritor 2: TERMINA escritura
```

```
justin@ubuntu-server:~/lab4$ cat BASE.txt
=== BASE DE DATOS ===
Registro 1: Ana
Registro 2: Luis
Registro del Escritor 1 - operacion: ESCRITURA
Registro del Escritor 2 - operacion: ESCRITURA
justin@ubuntu-server:~/lab4$
```

6. Pruebe un segundo escenario con cinco lectores y tres escritores. Compare su comportamiento con el escenario inicial.

```
justin@ubuntu-server:~/lab4$ python3 03_lectores_escritores.py 5 3
[ 0.20s] Lector 5: INICIA lectura (activos=1)
[ 0.80s] Lector 3: INICIA lectura (activos=2)
[ 0.99s] Lector 2: INICIA lectura (activos=3)
[ 1.16s] Lector 5: TERMINA lectura (3 lineas)
[ 1.27s] Lector 4: INICIA lectura (activos=3)
[ 1.33s] Lector 3: TERMINA lectura (3 lineas)
[ 1.49s] Lector 1: INICIA lectura (activos=3)
[ 1.73s] Lector 2: TERMINA lectura (3 lineas)
[ 2.05s] Lector 4: TERMINA lectura (3 lineas)
[ 2.18s] Lector 1: TERMINA lectura (3 lineas)
[ 2.18s] Escritor 3: INICIA escritura (ACCESO EXCLUSIVO)
[ 2.98s] Escritor 3: TERMINA escritura
[ 2.98s] Escritor 2: INICIA escritura (ACCESO EXCLUSIVO)
[ 3.97s] Escritor 2: TERMINA escritura
[ 3.97s] Escritor 1: INICIA escritura (ACCESO EXCLUSIVO)
[ 4.92s] Escritor 1: TERMINA escritura

--- Contenido final de BASE.txt ---
=== BASE DE DATOS ===
Registro 1: Ana
Registro 2: Luis
Registro del Escritor 3 - operacion: ESCRITURA
Registro del Escritor 2 - operacion: ESCRITURA
Registro del Escritor 1 - operacion: ESCRITURA

justin@ubuntu-server:~/lab4$ cat BASE.txt
=== BASE DE DATOS ===
Registro 1: Ana
Registro 2: Luis
Registro del Escritor 3 - operacion: ESCRITURA
Registro del Escritor 2 - operacion: ESCRITURA
Registro del Escritor 1 - operacion: ESCRITURA
justin@ubuntu-server:~/lab4$
```

Con 5 lectores y 3 escritores hubo más lectores solapados y los escritores esperaron más, porque cada uno debía aguardar a que salieran todos los lectores activos

La ejecución duró más que con 3 lectores y 2 escritores, pero BASE.txt quedó íntegro en ambos casos, con un registro por cada escritor

Actividad de consulta

Investigue sobre Barriers en Python

En programación concurrente, una barrera (Barrier) es un primitivo de sincronización que bloquea un grupo predeterminado de hilos hasta que un número específico de ellos alcanza el punto de control común

Elementos principales:

- Barrier(parties): crea la barrera para parties hilos
- wait(): el hilo se bloquea hasta que lleguen todos; luego todos se liberan a la vez
- parties: número de hilos que debe esperar
- n_waiting: cuántos hilos están esperando en ese momento
- abort() y reset(): cancelan o reinician la barrera

Diferencia con los semáforos: un semáforo controla el acceso o el turno; una barrera sincroniza a un grupo de hilos en un punto común

4.Preguntas de análisis

Responda con sus propias palabras:

1. ¿Por qué el orden de inicio de los hilos no basta para asegurar A1, A3, A2?

Porque start() solo deja el hilo listo para ejecutarse; quién corre primero lo decide el planificador del sistema operativo y puede cambiar en cada ejecución. Por eso el orden debe imponerlo la sincronización con semáforos, no el orden de las llamadas a start().

2. ¿Cómo garantizan los valores iniciales de los semáforos que A1 actúe primero?

s1 empieza en 1, así que A1 pasa su acquire() sin esperar. s3 y s2 empiezan en 0, por lo que A3 y A2 se bloquean en su acquire() hasta que alguien haga release() y asi, aunque A2 o A3 arranquen antes, deben esperar a A1

3. ¿Por qué cada agente debe habilitar al siguiente solo después de finalizar el acceso al LLM?

Porque si hiciera release() antes de terminar, el siguiente agente podría entrar mientras el primero aún usa el LLM y esto haría que haya dos accesos simultáneos y se perdería la exclusión y el orden

4. ¿Por qué pueden entrar varios lectores, pero únicamente un escritor?

Los lectores solo consultan el archivo y no lo modifican, por lo que pueden solaparse sin riesgo y un escritor sí cambia el contenido, así que si hubiera otro hilo al mismo tiempo se podrían leer datos incompletos o mezclar escrituras y por esta razón el escritor requiere acceso exclusivo

5. ¿Qué ocurriría si dos lectores modificaran el contador de lectores activos sin protegerlo?

Habría una condición de carrera ya que ambos podrían leer el mismo valor y sumar 1 cada uno, y el contador quedaría con un valor incorrecto y entonces podría bloquearse el archivo para siempre, si el contador nunca llega a 0, o permitir que un escritor entre mientras aún hay lectores

6. ¿Qué ocurriría si un escritor no liberara el semáforo de acceso al archivo?

El semáforo quedaría en 0 para siempre y ningún lector ni otro escritor podría entrar, y todos esperarían indefinidamente entonces el programa se quedaría bloqueado y por eso se usa try/finally, para garantizar el

release()

7. ¿La exclusión mutua por sí sola evita la inanición de escritores? Justifique su respuesta.

No. La exclusión mutua evita accesos simultáneos conflictivos, pero no decidequién entra primero y con esta solución, si llegan lectores de manera continua, el contador de lectores activos nunca baja a 0 y el escritor puede esperar indefinidamente y para evitarlo se necesita una política adicional, como un semáforo de turno que bloquee la entrada de nuevos lectores cuando hay un escritor esperando

5.Conclusión

En este laboratorio se comprobó que los semáforos permiten coordinar hilos en Python, tanto para imponer un orden de ejecución como para controlar el acceso a un recurso compartido y sin exclusión mutua aparecieron condiciones de carrera, mientras que con semáforos el contador y el archivo quedaron correctos En lectores y escritores, varios lectores pudieron leer a la vez y cada escritor accedió de forma exclusiva, manteniendo la integridad de BASE.txt y por lo tanto, la sincronización es esencial para que los hilos trabajen de forma segura y predecible

6.Anexos

Anexo: Uso de Inteligencia Artificial Generativa

Elemento	Descripción del Estudiante
Modelo Utilizado	**Google Gemini.**
Prompt	**"Explícame los conceptos teóricos sobre la sincronización de hilos con semáforos en Python y la estructura del problema de lectores-, sin resolver la práctica de laboratorio directamente."**
Resultado Obtenido	**Explicación conceptual sobre exclusión mutua, inicialización de semáforos en orden cíclico, manejo de concurrencia en lectura/escritura de archivos en Python y comandos para el entorno Ubuntu Server.**
Validación	**Se verificó la información teórica ejecutando los programas en Python (00_agentes_ia.py y 03_lectores_escritores.py) sobre el sistema operativo Ubuntu Server, corroborando en consola y en el archivo BASE.txt la correcta sincronización de los hilos.**

Referencias IEEE

- **[1] Python Software Foundation, "threading — Thread-based parallelism," *Python 3 Documentation*, 2026. [Online]. Available: https://docs.python.org/3/library/threading.html**
- **[2] Canonical, "Ubuntu Server documentation," *Ubuntu Documentation*, 2026. [Online]. Available: https://documentation.ubuntu.com/server/**
- **[3] J. Torres, "Sistemas Operativos," Universidad de La Laguna, OpenCourseWare (OCW). [Online]. Available: https://ull-esit-sistemas-operativos.github.io/ssoo-apuntes/ocw/**

- **[4] M. Aguirre, "Laboratorio V: Sincronización de hilos en Python," Escuela Politécnica Nacional, Quito, Ecuador, Período 2026-B**